🐳 Docker — Step 4

Multi-Tier Apps: a Next.js Frontend + a Node API + Redis, all wired with Compose

Project: `step4-prac` • An Emoji Reaction Board (Next.js → Express → Redis) • Generated 25 June 2026 •
WORK IN PROGRESS

About this guide. This covers what you learned in Stage 4 *so far* — building a three-tier app and making its containers talk to each other. It is intentionally a living document: there's more research and practice still to come (production builds, shipping, orchestration), and the last section maps out that road. Everything here is beginner-first: concepts explained from scratch, with the real errors you hit and why they happened.

📋 **Contents**

1. The Big Picture — what Stage 4 adds

Step 3 had two containers (app + Redis). **Step 4 adds a real frontend as its own service**, turning this into a classic **three-tier app**:

```
Browser ──▶ frontend (Next.js) ──▶ backend (Express) ──▶ redis
on your Mac    port 3000              port 4000          port 6379
```

```
└──────── all three on Docker's private network ────────┘
```

- **frontend** — Next.js: what the user sees and clicks (the UI).
- **backend** — Express: a JSON API. No HTML — just data.
- **redis** — stores the data so it survives restarts.

> 💡 **Why three tiers?** Real apps separate the *presentation* (frontend), the *logic/data access* (API), and the *storage* (database). Each can be built, scaled, and restarted on its own. Step 4 is that separation in miniature.

2. The #1 concept: the browser is NOT on the Docker network

This is the single most important idea in Stage 4. Containers can talk to each other by **service name** (e.g. `backend`, `redis`). But your **browser runs on your Mac** — it is *outside* Docker's private network, so it **cannot** reach `http://backend:4000`.

> 🚨 **The trap:** beginners try to call the API from browser code using the service name and it fails — because only *containers* can resolve service names, not the browser.

How Next.js solves it: the Backend-For-Frontend (BFF) pattern

Next.js has **two halves**: a *server* (runs in the container) and the *browser* code it sends out. We route everything through the server:

```
1. Browser calls /api/reactions  ←  same-origin, no CORS needed
2. Next.js server (in the container, ON the network) receives it
3. The server calls http://backend:4000  ←  service name works here!
4. backend asks redis, answer flows back up
```

> 💡 **In one line:** the browser only ever talks to Next.js; the Next.js *server* is the bridge that reaches the API over the Docker network. That's the BFF pattern, and it's how real Next.js apps work.

3. The services & files

```
step4-prac/
├── docker-compose.yml        # wires the 3 services together
├── .env                      # config: API_URL, REDIS_URL
├── backend/                  # the Express JSON API
│   ├── index.js              # GET/POST endpoints → Redis
│   ├── package.json
│   └── Dockerfile            # node:20-alpine
└── frontend/                 # the Next.js app
    ├── src/app/page.js               # the UI (runs in browser)
    ├── src/app/api/reactions/route.js# the SERVER bridge to backend
    ├── package.json
    └── Dockerfile            # node:20-slim (see §6 for why)
```

docker-compose.yml (the heart of it)

```
services:
  frontend:
    build: ./frontend
    ports: ["3000:3000"]            # browser reaches Next.js here
    environment:
      API_URL: ${API_URL}           # = http://backend:4000
      WATCHPACK_POLLING: "true"     # reliable hot reload in Docker
    depends_on: [backend]
    volumes:
      - ./frontend:/app             # live code
      - /app/node_modules           # protect container's modules
      - /app/.next                  # protect Next's build cache

  backend:
    build: ./backend
    environment:
      REDIS_URL: ${REDIS_URL}       # = redis://redis:6379
    command: npm run dev            # nodemon live reload
    depends_on: [redis]
    volumes:
      - ./backend:/app
      - /app/node_modules
    # no published port — only the Next.js server calls it, internally

  redis:
    image: redis:7
    volumes: ["redis-data:/data"]   # persistence

volumes:
  redis-data:
```

> ⚠️ **Service name must match.** The service is called `backend`, so `API_URL` must be `http://backend:4000` — not `http://api:4000`. A mismatch gives `ENOTFOUND backend`. (This was a real bug we fixed.)

4. Service-to-service communication

Inside Docker's private network, every service is reachable by its **name** as if it were a hostname. Docker runs an internal DNS that turns the name into the right address.

```
// frontend/src/app/api/reactions/route.js  — runs on the SERVER
const apiUrl = process.env.API_URL || "http://backend:4000";

export async function GET() {
  const res = await fetch(`${apiUrl}/reactions`, { cache: "no-store" });
  return Response.json(await res.json());
}
```

Who calls whom	Address used	Why it works
browser → Next.js	`/api/reactions`	same origin (same site), no network hop outside
Next.js server → backend	`http://backend:4000`	service name on the Docker network
backend → redis	`redis://redis:6379`	service name on the Docker network

5. `RUN` vs `CMD` (a bug you debugged)

These two Dockerfile keywords run at **completely different times** — confusing them caused a real error for you.

	`RUN`	`CMD`
When	while **building** the image	when a **container starts**
Purpose	prepare the image (e.g. `npm install`)	run your app
Can be overridden?	no (already happened)	yes — via compose `command:`

> 🚨 **The bug:** writing `RUN ["node","index.js"]` tried to start the server *during the build* — before Redis existed — giving `ENOTFOUND redis` and a long hang. The fix: it must be `CMD`, so

the app starts at container-run time when Redis is up.

💡 **Rule of thumb:** `RUN` = build the image. `CMD` = run the app. Your app-starting line is always `CMD` . Tip: if the build log prints `RUN ["node",...]` , your file still says RUN — a correct `CMD` never appears as a build step.

6. Base images: Alpine vs Slim (the SWC saga)

The word after `node:20-` picks which Linux your image is built on. This caused a real headache with Next.js.

Image	Linux	C library	Size	Best for
`node:20`	Debian (full)	glibc	~1 GB	rarely (too big)
`node:20-slim`	Debian (stripped)	**glibc**	~200 MB	**Next.js**
`node:20-alpine`	Alpine	musl	~130 MB	plain Node (your backend)

What went wrong on Alpine

Next.js compiles your code with **SWC**, a *native binary* (built per-OS). Alpine uses **musl** instead of the standard **glibc**, so `npm install` often fails to install the right SWC binary. Next.js then tries to **download it at runtime** — slow and flaky:

```
Downloading swc package @next/swc-linux-x64-musl... to /root/.cache/next-swc
```

⚠️ **Why it never finished:** that download lands in `/root/.cache` — the container's throwaway layer, not a volume. Every rebuild starts a fresh container with an empty cache, so it re-downloads from zero each time.

💡 **The fix:** use `node:20-slim` for the frontend. With glibc, the SWC binary installs cleanly *during the build* (baked into the image) — no runtime download at all. Keep `node:20-alpine` for the plain-Node backend; it has no such native-binary drama.

7. `.env` — who can see which variable

The `.env` file works in **two layers**, and this trips people up:

1. **Compose reads** `.env` to fill `${...}` placeholders in `docker-compose.yml`. At this point nothing is inside any container yet.
2. **A variable reaches a container only if you list it under that service's** `environment:` (or `env_file:`). It is per-service — containers do *not* automatically share variables.

Variable	frontend sees it?	backend sees it?
`API_URL`	✅ (in frontend's `environment:`)	❌
`REDIS_URL`	❌	✅ (in backend's `environment:`)

🚨 **Next.js gotcha:** in the frontend, only the **server** can read `process.env.API_URL`. The **browser** can't — unless the name starts with `NEXT_PUBLIC_`. This is a safety feature so secrets don't leak to the browser. Keep `API_URL` server-only (no `NEXT_PUBLIC_`).

⚠️ **Changing `.env` needs a recreate.** Env vars are set when a container *starts*, not hot-reloaded. After editing `.env`, run `docker compose up -d` (it recreates the affected service). A plain `restart` won't pick up new values.

8. Installing libraries: restart vs rebuild

Does `docker compose restart frontend` install a new library? **No.** `restart` only restarts the process — it never runs `npm install` or rebuilds.

What you do	New library works?
`docker compose restart frontend` (only)	❌ nothing installed
`npm install X` on your **Mac** + restart	❌ hidden by the anonymous volume (see below)
`docker compose exec frontend npm install X`	✅ installed in the container's volume
add to `package.json` → `up --build`	✅ baked into the image (permanent)

🚨 **The volume trap.** Because of `- /app/node_modules`, the container's modules live in an anonymous volume that *hides* your Mac's `node_modules`. So installing on your Mac does nothing for the container — the install must happen **inside the container** (`exec ... npm install`) or via a **rebuild**.

💡 **Two clean workflows:** (A) quick dev add → `docker compose exec frontend npm install X`, then `restart frontend` if Next doesn't pick it up. (B) permanent → ensure it's in `package.json`, then `docker compose up --build`. Do (A) for speed, (B) now and then to keep the image in sync.

9. What we built: the Emoji Reaction Board

An interactive board where clicking an emoji bumps a live counter — demonstrating the full chain with a real **POST** (not just GET).

Backend — two endpoints, one Redis hash

```
app.use(express.json());          // parse JSON request bodies

// All counts live in ONE Redis HASH: reactions = { fire: 12, love: 30, ... }
app.get("/reactions", async (req, res) => {
  const raw = await redisClient.hGetAll("reactions");
  const counts = {};
  for (const k of ["fire","love","wow","nice"]) counts[k] = Number(raw[k]||0);
  res.json(counts);
});

app.post("/reactions", async (req, res) => {
  const { emoji } = req.body;
  if (!["fire","love","wow","nice"].includes(emoji))  // allow-list = validation
    return res.status(400).json({ error: "unknown reaction" });
  const count = await redisClient.hIncrBy("reactions", emoji, 1); // atomic +1
  res.json({ emoji, count });
});
```

New concept	What it taught
Redis **hash** (`hIncrBy` , `hGetAll`)	one key holding many named counters (vs a single integer)
POST with a JSON body	sending data *up* the chain, not just reading
`express.json()`	middleware to parse the request body
Atomic `hIncrBy`	safe even if many people click at the same instant
Allow-list validation	reject unknown input (`banana` → 400 error)
Optimistic UI update	bump the number instantly, then sync to Redis's real value

10. Watching logs in separate terminals

To follow each service live, run one command per terminal:

```
docker compose logs -f frontend      # terminal 1
docker compose logs -f backend       # terminal 2
docker compose logs -f redis         # terminal 3
```

- `-f` = **follow** (stream new lines as they happen). `Ctrl+C` stops watching (not the container).
- `--tail 20` limits backlog; `-t` adds timestamps.
- All at once in one terminal: `docker compose logs -f` (color-coded, prefixed).

11. Common errors you hit & their fixes

Error / symptom	Cause	Fix
`ENOTFOUND redis` during **build**	used `RUN` instead of `CMD` — app started at build time	change last line to `CMD ["node","index.js"]`
`ENOTFOUND backend` at runtime	`API_URL` didn't match the service name	set `API_URL=http://backend:4000`
SWC download hangs forever	Alpine/musl needs runtime SWC download	switch frontend to `node:20-slim`
`ECONNRESET` / `npm network aborted`	internet dropped mid-download	retry `up --build` (resumes from cache); restart Docker Desktop
New library "module not found" after `restart`	`restart` doesn't install anything	`exec ... npm install` or `up --build`
`requires at least 2 arg(s)`	`exec` needs a service *and* a command	`docker compose exec backend sh`

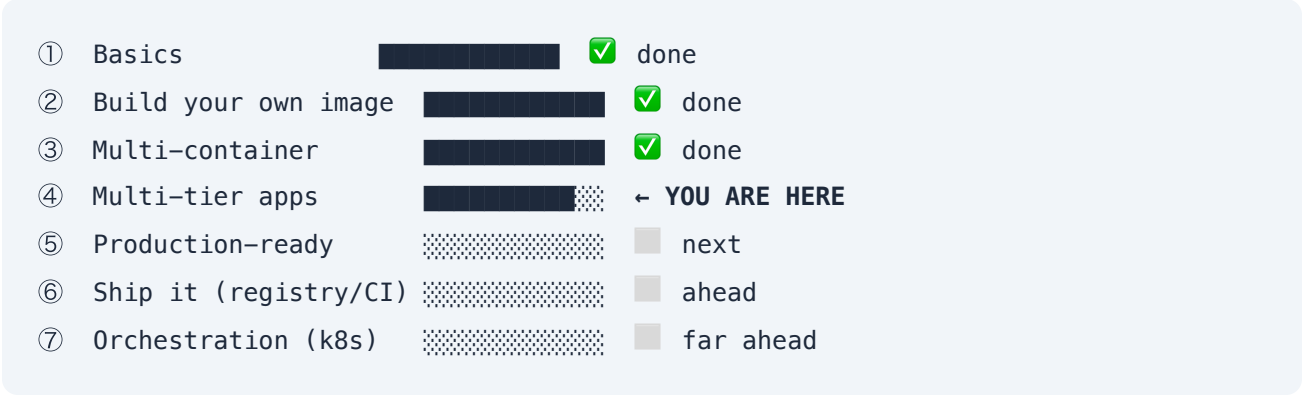
12. Commands cheat-sheet

```
# Lifecycle
docker compose up --build         # rebuild images, then start
docker compose up -d              # start in background (also applies .env changes)
docker compose restart frontend   # restart ONE service's process (no install/rebuil
docker compose down               # stop & remove containers + network
docker compose down -v            # ...also remove volumes (wipes Redis data!)

# Inspect
docker compose ps                 # list this project's containers
docker compose logs -f backend    # follow one service's logs

# Inside containers
docker compose exec -it frontend sh             # open a shell
docker compose exec frontend npm install X      # install a lib in the container
docker compose exec redis redis-cli hgetall reactions   # see the Redis hash
```

13. Where you are & the road ahead



You're a solid **intermediate** — you can containerize and run a real multi-service app locally. Roughly **~65–70%** of the Docker a developer needs day-to-day; less of the full production+orchestration world (which most people learn on the job).

Stage 5 — Production-ready (your natural next step)

- **Multi-stage builds** → a tiny production image (Next.js `standalone` output)
- **Healthchecks** + `depends_on: condition: service_healthy` (wait until Redis is truly ready)
- Run as a **non-root user**, restart policies, resource limits
- A **dev/prod compose split** (`docker-compose.override.yml`)

Stage 6 — Shipping it

- Image **tags**, pushing to a **registry** (Docker Hub / ECR)
- **CI/CD** that builds, tests, and deploys automatically

Stage 7 — Orchestration (the deep end)

- **Kubernetes** (or Swarm): scaling, secrets, self-healing, observability

> 💡 **Good news:** Stages 5–6 build on what you already know — they make your current app production-grade, not brand-new concepts. Finish Stage 5 on this very app and you'll jump to ~80% of "developer Docker." Stage 7 is a new world; you don't need it until you deploy at scale.

14. Glossary

Term	Meaning
Three-tier app	frontend (UI) + backend (API) + database, as separate services
BFF (Backend-For-Frontend)	the frontend's server makes the API calls on the browser's behalf
Service discovery	reaching a container by its service name as a hostname
Route handler	a Next.js `route.js` that runs on the server (GET/POST)
Client component	React code marked `"use client"` that runs in the browser
glibc / musl	the two C libraries; Debian uses glibc, Alpine uses musl
Native binary	a program compiled per-OS (e.g. SWC) — not portable like JS
Redis hash	one key holding many named fields/counters
Atomic operation	completes fully without interruption — safe under concurrency
Anonymous volume	unnamed Docker storage used to protect a container-only folder